DevOps Documentation for API application

Purpose: this document outlines the steps needed for initial setup, deployment, architecture and git flow.

GIT FLOW

INFRASTRUCTURE ARCHITECTURE AND WORKFLOW

```
flowchart TD
    Client["Client<br/>(Postman, Browser, curl)"]
    Gateway["API Gateway"]
    Lambda["Lambda<br/>(main.lambda_handler)"]
    S3["S3<br/>(for query results)"]
    RDS["RDS<br/>(PostgreSQL inside private subnet)"]

    Client -->|HTTPS Request| Gateway
    Gateway -->|Triggers| Lambda
    Lambda -->|Stores data| S3
    Lambda --> RDS
```

AWS RESOURCES

```
graph TD
  subgraph VPC
    IGW[Internet Gateway]
    NAT[NAT Gateway]
    Pub1[Public Subnet]
    Pub2[Public Subnet]
    Priv1[Private Subnet]
    Priv2[Private Subnet]
    LambdaFn[AWS Lambda]
```

```
  RDS[(RDS - PostgreSQL)]
end

IGW --> Pub1
IGW --> Pub2
Pub1 --> NAT
Pub2 --> NAT
NAT --> Priv1
NAT --> Priv2
LambdaFn -->|Connects via ENI| Priv1
LambdaFn --> RDS
LambdaFn --> S3[(S3 - Query Storage)]
```

Initial setup steps:
1. Create a GitHub Actions role in AWS so that the runner can create/destroy/modify resources:
name: GitHubActionsRole
permissions:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAllActions",
      "Effect": "Allow",
      "Action": [
        "s3:*",
        "ecr:*",
        "lambda:*",
        "apigateway:*",
        "iam:*",
        "logs:*",
        "secretsmanager:*",
        "ec2:*",
        "rds:*"
      ],
```

```
            "Resource": "*"
        }
    ]
}
```

2. Create an OIDC Provider

   token.actions.githubusercontent.com

3. Update the role above with this trust relationship so that the role can be used by GitHub Actions

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "Federated": "arn:aws:iam::<ACCOUNT ID>:oidc-provider/token.acti
            },
            "Action": "sts:AssumeRoleWithWebIdentity",
            "Condition": {
                "StringEquals": {
                    "token.actions.githubusercontent.com:aud": "sts.amazonaws.com
                },
                "StringLike": {
                    "token.actions.githubusercontent.com:sub": "repo:<repo owner>/
                }
            }
        }
    ]
}
```

Now Github actions can create resources in AWS, the policy can be adjusted per needs.

For Terraform we will do follow an Enterprise module approach, so we will have

a terraform folder which will contain the following layout:

```
tree
.
├── config
│   ├── dev.tfvars
│   └── qa.tfvars
├── environments
│   ├── dev
│   │   ├── backend.tf
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
│   ├── prod
│   └── qa
├── modules
│   ├── api_gateway
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
│   ├── ecr
│   ├── iam
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
│   ├── lambda
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
│   ├── rds
│   │   ├── main.tf
│   │   ├── output.tf
│   │   └── variables.tf
│   ├── s3
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
```

```
│   └── vpc
│       ├── main.tf
│       ├── outputs.tf
│       └── variables.tf
└── plans
```

This means we can have separate configs per environment and the modules will be easier to maintain and update.

CI-CD WORKFLOW

```
flowchart TB
    subgraph "Git Branches"
        development["development branch"]
        qa["qa branch"]
        prod["prod branch"]
    end

    subgraph "Pipeline Triggers"
        push_dev["Push to development\n(Changes to app/terraform)"]
        pr_to_qa["Pull Request to qa\nfrom development"]
        pr_to_prod["Pull Request to prod\nfrom qa"]
        manual["Manual Trigger\n(workflow_dispatch)"]
    end

    subgraph "Environment Selection"
        det_env["Determine Environment\n- dev (from development)\n- qa (from
    end

    subgraph "File Change Detection"
        detect_changes["Detect File Changes\n- terraform\n- lambda"]
    end

    subgraph "Infrastructure Jobs"
        tf_plan["Terraform Plan"]
        tf_apply["Terraform Apply"]
        tf_destroy["Terraform Destroy\n(Manual Only)"]
    end
```

```
subgraph "Application Jobs"
   build_lambda["Build Lambda Package"]
   update_lambda["Update Lambda\nwith New Package"]
end

%% Connect git branches to triggers
development --> push_dev
development --> pr_to_qa
qa --> pr_to_prod

%% Connect triggers to environment selection
push_dev --> det_env
pr_to_qa --> det_env
pr_to_prod --> det_env
manual --> det_env

%% Connect environment selection to change detection
det_env --> detect_changes

%% Connect change detection to jobs
detect_changes -->|terraform changes| tf_plan
detect_changes -->|lambda changes| build_lambda

%% Terraform job flow
tf_plan --> tf_apply
manual -->|destroy_infrastructure=true| tf_destroy

%% Lambda job flow
build_lambda --> update_lambda
tf_apply --> update_lambda

%% Special styles with better readability
classDef branch fill:#ffe6ff,stroke:#333,stroke-width:2px,color:#000
classDef trigger fill:#e6e6ff,stroke:#333,stroke-width:1px,color:#000
classDef infraJob fill:#e6ffe6,stroke:#333,stroke-width:1px,color:#000
classDef appJob fill:#fff2cc,stroke:#333,stroke-width:1px,color:#000
```

```
class development,qa,prod branch
class push_dev,pr_to_qa,pr_to_prod,manual trigger
class tf_plan,tf_apply,tf_destroy infraJob
class build_lambda,update_lambda appJob
```

The CI/CD flow works as follows:

```yaml
- name: Check for file changes
  id: filter
  uses: dorny/paths-filter@v2
  with:
    filters: |
      terraform:
        - 'terraform/**'
        - '.github/workflows/ci-cd.yml'
      lambda:
        - 'Dockerfile'
        - 'requirements.txt'
        - 'docker-compose.yml'
        - 'app/**'
      bootstrap:
        - 'terraform/bootstrap/**'
```

- the pipeline checks for changes in 3 main places: terraform, lambda and bootstrap
- if any change into the specific subsections, then it will trigger the flow for that particular pipeline. This will ensure that we only ‘build’ what we need
- automatic deployment is only done for development branch, for QA we need a PR from development branch and 1 approval, for production environment we need a PR with at least 2 approvals.
- the pipeline also features a code scan process for the files inside the ./app/ folder and if that returns failures, then the lambda zip package step would not be executed

```
- name: Run Python security scan (Bandit)
  id: bandit-scan
  run: |
    echo "🔐 Scanning Python files in ./app/ for security issues..."
    pip install bandit
    bandit -r ./app -f json -o bandit_report.json || true
    cat bandit_report.json

    ISSUE_COUNT=$(jq '.results | length' bandit_report.json)
    echo "🚨 Issues found: $ISSUE_COUNT"

    if [ "$ISSUE_COUNT" -gt 0 ]; then
      echo "scan_passed=false" >> $GITHUB_OUTPUT
      echo "❌ Security issues detected in Python code."
    else
      echo "scan_passed=true" >> $GITHUB_OUTPUT
      echo "✅ No security issues found."
    fi
```

STATEFILE AND LOCKTABLE BOOTSTRAPPING

For best practices we want to store the state lock file in a DynamoDB table so we dont have multiple users sharing it at the same time. The bootstrap step makes sure that the table is created before anything else runs, by having it's own bootstrap module.

For the statefile, which we store it in its own S3 bucket, the pipeline checks if the bucket is created, if not it creates it, so the following steps can place the statefile in the S3. Then the pipeline updates the ${env}/backend.tf is being populated with appropriate paths (i.e. dev):

```
terraform {
  backend "s3" {
    bucket         = "terraform-state-bmoldo-devops-home-task"
    key            = "terraform/dev/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-locks-dev"
  }
}
```

Local Terraform infrastructure deployment guide

MAKEFILE ADDITION

To combat the potential downtime of Github Actions we have this Makefile which allows us to make deployments even if GitHub actions is down. Here is a breakdown of the Makefile and how to use it:

Prerequisites

- AWS CLI configured with appropriate permissions
- Terraform installed
- Make utility
- GitHub repository set up

Configuration

The Makefile uses several variables that can be customized:

Variable	Description	Default

ENV	Target environment (dev, prod, etc.)	dev
ACCOUNT_ID	AWS account ID	070503547773
REPO_OWNER	GitHub repository owner	your-github-owner
REPO_NAME	GitHub repository name	your-repository-name
API_URL	API endpoint for healthchecks	None (required for healthcheck)
TF_LOG	Terraform log level	error

Directory Structure

```
.
├── terraform/
│   ├── bootstrap/       # Bootstrap Terraform code
│   ├── config/          # Environment-specific tfvars
│   │   ├── dev.tfvars
│   │   └── prod.tfvars
│   ├── environments/    # Environment-specific Terraform code
│   │   ├── dev/
│   │   └── prod/
│   └── plans/           # Generated plan files
├── placeholder_lambda/   # Sample Lambda code (generated)
└── Makefile
```

Available Commands

Basic Workflow

`make init ENV=dev` - Initialize environment and sync state

`make lambda-package ENV=dev` - Create Lambda deployment package and upload to S3

`make plan ENV=dev` - Plan Terraform changes

`make apply ENV=dev` - Apply Terraform changes

All Available Commands

Command	Description
`make init`	Initialize environment and sync remote state

`make bootstrap`	Set up S3 bucket and DynamoDB table for Terraform state
`make lambda-package`	Create a sample Lambda deployment package and upload to S3
`make plan`	Generate and save Terraform plan
`make apply`	Apply Terraform changes
`make destroy`	Destroy all resources
`make healthcheck API_URL=https://your-api.com`	Perform healthcheck on deployed API
`make clean`	Clean up generated files (if implemented)

Environment Management

The Makefile supports multiple environments through the `ENV` variable:

```
make plan ENV=prod
make apply ENV=prod
```

State Management

The infrastructure state is stored in S3 with DynamoDB locking:

- S3 Bucket: `terraform-state-$(REPO_OWNER)-$(REPO_NAME)`
- State Key: `terraform/$(ENV)/terraform.tfstate`
- DynamoDB Lock Table: `terraform-locks-$(ENV)`

Lambda Deployment

A placeholder Lambda function is automatically created and uploaded to:

- S3 Bucket: `lambda-packages-$(ENV)-$(ACCOUNT_ID)`
- S3 Key: `lambda/lambda_deployment_package.zip`

Getting Started

1️⃣ Set your repository and AWS account information:

```
export REPO_OWNER=your-github-username
export REPO_NAME=your-repo-name
export ACCOUNT_ID=your-aws-account-id
```

2️⃣ Bootstrap the Terraform backend infrastructure:

`make bootstrap ENV=dev`

3️⃣ Initialize the Terraform environment:

`make init ENV=dev`

4️⃣ Create and upload a Lambda package:

`make lambda-package ENV=dev`

5️⃣ Plan and apply your infrastructure:

`make plan ENV=dev`

`make apply ENV=dev`

6️⃣ Verify your deployment:

`make healthcheck API_URL=https://your-api-endpoint`

Notes

- The Makefile automatically generates backend configuration for each environment
- Remote state is synchronized during initialization
- Terraform variables are loaded from environment-specific `.tfvars` files
- A placeholder Lambda function is created if you don't have your own Lambda code

Alternative Pipeline Option

As an alternative to a single pipeline solution, we can use child pipelines. In the repo, inside `.github` we have the following setup:

```
├── CODEOWNERS
├── separate-workflows
│   ├── actions
│   │   └── setup-aws-terraform
│   │       └── action.yml
│   ├── infrastructure-pipeline.yml
│   ├── lambda-workflow.yml
│   ├── python-scan.yml
│   └── terraform-workflow.yml
```

The separate workflows follow a similar pattern: check for specific folder changes → trigger the appropriate pipeline file.

Additional suggested improvements:

- output the tf plan result to Slack
- output the code scan results to Slack and/or to an S3 bucket
- when PR is created send notification to Slack
- dynamic analysis for the healthcheck
- add custom dns name for the api